

Q1. The two problems are **data redundancy** and **data inconsistency**. The same customer address is stored in multiple places, creating duplicate data, and updating only one copy causes departments to have conflicting information. A database provides a shared, centralized source of data so both departments access the same customer record. Controlled updates help ensure that changes are consistently reflected.

Q2.

1. **Data integrity and constraints** — A DBMS can prevent invalid data such as an account having a negative balance. Without this, incorrect banking data could be stored.
2. **Concurrency control** — A DBMS manages simultaneous transactions from ATMs and banking applications. Without it, concurrent transactions could produce incorrect account balances.
3. **Transaction recovery** — A DBMS can recover committed data after failures. Without it, completed banking transactions could be lost after a system crash.

Q3.

Formal Term	Everyday Term	ICICI Bank Example
Relation	Table	Customer table
Tuple	Row/record	One Customer's record
Attribute	Column/field	Customer id
Degree	Number of columns	Number of columns in customer
Cardinality	Number of rows	Number of customers

Q4.

1. customer.customer_id → Primary key
2. account.account_number → Candidate key
3. account.customer_id → Foreign key
4. account.balance → None
5. employee.manager_id → Foreign key
6. branch.ifsc_code → Candidate key

Q5.

1. OLTP — A balance check is a small, real-time operational transaction.
2. OLAP — Calculating quarterly non-performing loans involves analysing a large amount of historical data.
3. OLTP — An ATM withdrawal is a real-time banking transaction.

4. OLAP — It requires analysing transactions over a large historical period.
5. OLTP — Opening an account is an operational transaction that changes banking data.
6. OLAP — Marketing is analysing customer data to identify patterns and rankings.

Q6.

Running a quarterly regulatory report directly against the core banking database can consume CPU, memory, I/O and database connections needed for real-time transactions. Large analytical queries can also increase contention and affect the response time of operational workloads. In addition, reporting workloads are different from transactional workloads and may require historical aggregation. I would offer a **separate reporting/analytical database or data warehouse**, populated from the core system.

Q7.

Atomicity → d. Money is debited but never credited

Consistency → c. An account exists for a customer who does not

Isolation → b. Two ATMs both pay out against the same balance

Durability → a. A committed transfer disappears after a power cut

Q8.

The write-ahead log is written to disk before the actual data page so that PostgreSQL has a durable record of the change before acknowledging the transaction. If the system crashes before the data page is written, PostgreSQL can use the WAL to recover the committed change. If the data page were written first without the WAL being safely stored, a crash could leave the database without a reliable record needed for recovery.

Q9.

SELECT * creates a dependency on the physical structure of the table and therefore breaks **logical data independence**. If the bank later adds a pan_number column, the query automatically returns an additional column, which can break application code that expects a fixed number or order of columns. The application should explicitly select only the columns it requires.

Q10.

Oracle → Core banking systems requiring enterprise-scale relational transactions and existing Oracle infrastructure.

PostgreSQL → Applications requiring a capable open-source relational database with strong SQL and transactional support.

MySQL → Applications requiring a widely used relational database for suitable transactional workloads.

Redis → Fast in-memory caching and low-latency access to frequently accessed data.

Q11.

```
SELECT current_database(),  
       current_user,  
       inet_server_port();
```

Q12.

```
SELECT column_name,  
       data_type,  
       is_nullable  
FROM information_schema.columns  
WHERE table_name = 'loan'  
ORDER BY ordinal_position;
```

Then

```
SELECT COUNT(*) AS number_of_columns  
FROM information_schema.columns  
WHERE table_name = 'loan';
```

Q13.

```
SELECT 'branch' AS table_name, COUNT(*) AS row_count FROM branch  
UNION ALL
```

```

SELECT 'customer', COUNT(*) FROM customer
UNION ALL
SELECT 'account', COUNT(*) FROM account
UNION ALL
SELECT 'account_txn', COUNT(*) FROM account_txn
UNION ALL
SELECT 'loan', COUNT(*) FROM loan
UNION ALL
SELECT 'loan_payment', COUNT(*) FROM loan_payment
UNION ALL
SELECT 'card', COUNT(*) FROM card
UNION ALL
SELECT 'beneficiary', COUNT(*) FROM beneficiary
UNION ALL
SELECT 'employee', COUNT(*) FROM employee
ORDER BY table_name;

```

Q14.

```

SELECT tc.table_name AS child_table,
       kcu.column_name AS child_column,
       ccu.table_name AS parent_table,
       ccu.column_name AS parent_column
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
  ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage ccu
  ON ccu.constraint_name = tc.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY'
ORDER BY tc.table_name;

```

Q15.

```
SELECT relname AS table_name,  
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size  
FROM pg_catalog.pg_statio_user_tables  
ORDER BY pg_total_relation_size(relid) DESC  
LIMIT 3;
```

Q16.

INSERT CUSTOMER

```
INSERT INTO customer  
(customer_id, first_name, last_name, date_of_birth, email, phone,  
 city, kyc_status, risk_rating, customer_since, home_branch_id)  
VALUES  
(1032, 'Latha', 'Krishnan', DATE '1985-09-23',  
 'latha.krishnan@example.com', '9845099887',  
 'Chennai', 'VERIFIED', 'MEDIUM',  
 DATE '2026-03-01', 5);
```

INSERT ACCOUNT

```
INSERT INTO account  
(account_id, account_number, customer_id, branch_id, account_type,  
 balance, currency, status, opened_date, interest_rate)  
VALUES  
(9047, 'ICIC0000009047', 1032, 5, 'CURRENT',  
 500000.00, 'INR', 'ACTIVE',  
 DATE '2026-03-01', 0.00);
```

Reading the values for both tables

```
SELECT *  
FROM customer  
WHERE customer_id = 1032;
```

```
SELECT *  
FROM customer  
WHERE customer_id = 1032;
```

Q17.

```
SELECT *  
FROM account  
WHERE account_id = 9047;
```

```
UPDATE account  
SET balance = balance - 20000  
WHERE account_id = 9047;
```

```
SELECT account_id,  
       account_number,  
       balance  
FROM account  
WHERE account_id = 9047;
```

Q18.

```
DELETE FROM customer
```

```
WHERE customer_id = 1032;
```

```
DELETE FROM account
```

```
WHERE customer_id = 1032;
```

```
DELETE FROM customer
```

```
WHERE customer_id = 1032;
```

```
SELECT COUNT(*) FROM customer;
```

```
SELECT COUNT(*) FROM account;
```

Q19.

Invalid account type

```
INSERT INTO account
```

```
(account_id, account_number, customer_id, branch_id, account_type,
```

```
balance, currency, status, opened_date, interest_rate)
```

```
VALUES
```

```
(9998, 'TEST0000009998', 1001, 1, 'FIXED',
```

```
1000.00, 'INR', 'ACTIVE', DATE '2026-03-01', 0.00);
```

Negative transaction amount

```
INSERT INTO account_txn
```

```
(txn_id, account_id, txn_date, txn_type, amount,
```

```
channel, narration, status, reference_no)
```

```
VALUES
```

```
(999998, 9001, TIMESTAMP '2026-03-01 10:00:00',
```

```
'DEBIT', -5000, 'ATM', 'test', 'SUCCESS', 'REF998');
```

Invalid KYC status

```
INSERT INTO customer
```

```
(customer_id, first_name, last_name, date_of_birth, email, phone,
```

```
city, kyc_status, risk_rating, customer_since, home_branch_id)
```

VALUES
(9999, 'Test', 'Customer', DATE '1990-01-01',
'test@example.com', '9999999999',
'Mumbai', 'IN_PROGRESS', 'LOW',
DATE '2026-03-01', 1);

Q20.

SESSION A

BEGIN;
UPDATE account
SET balance = balance - 5000
WHERE account_id = 9002;

SELECT balance
FROM account
WHERE account_id = 9002;

SESSION B

SELECT balance
FROM account
WHERE account_id = 9002;

SESSION A

ROLLBACK;

Q21.

SELECT account_id, balance
FROM account
WHERE account_id IN (9004, 9010)
ORDER BY account_id;

BEGIN;

UPDATE account

SET balance = balance - 12000

WHERE account_id = 9004;

UPDATE account

SET balance = balance + 12000

WHERE account_id = 9010;

UPDATE account

SET balance = balance + 12000

WHERE account_id = 9010;

SELECT account_id, balance

FROM account

WHERE account_id IN (9004, 9010)

ORDER BY account_id;

ROLLBACK;

SELECT account_id, balance

FROM account

WHERE account_id IN (9004, 9010)

ORDER BY account_id;

Q22 .

SHOW shared_buffers;

SHOW work_mem;

SHOW wal_level;

SHOW fsync;

shared_buffers → PostgreSQL's shared memory used for caching database pages.

work_mem → Memory available for individual query operations such as
sorting and hashing.

wal_level → Controls the amount/type of information written to the
write-ahead log.

fsync → Controls whether PostgreSQL ensures WAL/data is flushed to
durable storage before acknowledging certain writes.

Q23.

I would not use MyISAM for a UPI transaction system because it does not provide the transactional and crash-recovery guarantees required for financial transactions. A failure during a payment could leave the transaction state unreliable. I would use a transactional storage engine such as **InnoDB**, which provides transaction support and is appropriate for workloads requiring reliable updates.

Q24.

The account transaction workload has two different access patterns. Branch operations need fast access to transactions for one particular account, while the risk team needs large-scale scans and aggregations across many accounts and months of history. I would use the operational database for account-level transactional access and a separate analytical system/data warehouse for large historical aggregations. The programme's later modules build the corresponding database and analytical capabilities; the question specifically asks you to name the relevant programme modules after studying them.

Q25.

An application log saying "transfer successful" only proves that the application reached that point in its own execution; it does not prove that the database transaction was successfully committed. The actual evidence would be a successful database commit together with verification that the debit and credit were both persisted. Transaction records and the resulting account balances should also confirm the transfer.

Q26.

I would not set fsync = off on the core banking PostgreSQL server because it weakens durability and can cause committed data to be lost or corrupted after a system failure. Faster batch processing is not sufficient justification for accepting that risk on a system holding banking transactions. If it were used at all, it should be limited to a disposable development or temporary test environment where data loss is acceptable and no production banking data is involved

Q27.

First, keeping balance ≥ 0 in the database protects the rule regardless of which application, service or user writes to the database. Second, the database can enforce the constraint consistently even if a developer accidentally forgets to check the rule in application code. The application layer is more appropriate for rules that depend on complex external context or user-interface behaviour, such as deciding how to display a warning or collecting additional information before allowing a request.

Q28.

Four differences that could cause migration problems are **SQL dialect differences, database/schema terminology, command-line tools, and database-specific features and configuration**. PostgreSQL and Oracle use different SQL syntax and administrative commands, and concepts such as databases and schemas are organized differently. PostgreSQL commonly uses psql, while Oracle uses sqlplus. Features such as data types, sequences, functions, transaction behaviour and configuration parameters would also need to be tested during migration.

Q29.

Subject: Response to Ticket ICICI-4417

Moving from spreadsheets to a relational database will give the bank a shared and controlled source of customer and account information.

It will reduce duplicate copies of the same information across departments.

When customer information changes, authorised users can work with the same stored record.

The database can also enforce rules that prevent invalid account and transaction data from being stored.

It can control simultaneous updates when multiple banking systems access the same account.

Transactions can be completed as a single unit so that incomplete transfers do not leave inconsistent balances.

Committed information can also be recovered after a system failure.

However, a database does not automatically make incorrect business processes or incorrect input correct.

The bank still needs proper access controls, testing, monitoring and application validation. Staff must also follow controlled procedures instead of maintaining separate spreadsheet copies.

Applications should use transactions correctly and avoid bypassing database rules.

Regular backups, monitoring and recovery procedures must also be m